QuakeGuard

Distributed Earthquake Early Warning System

Technical Architecture & Protocol Specification

Release: **v2.0.0** (Hybrid Infrastructure & GNSS Automation)

Core Maintainers: @GiZano, @riccardo0731

Table of Contents

1. System Architecture & Overview

QuakeGuard is a distributed, high-throughput backend system designed for the real-time ingestion, cryptographic validation, and processing of seismic data from IoT devices[cite: 1]. It serves as the core infrastructure for an Earthquake Early Warning (EEW) system[cite: 1]. The architecture is explicitly designed to handle high-concurrency event firehosing during seismic swarms while maintaining strict security boundaries.

1.1. High-Level Topology

The infrastructure is decoupled into three primary tiers:

- **Edge Layer (IoT):** Composed of ESP32-C3 SuperMini microcontrollers interfaced with ADXL345 digital accelerometers[cite: 1]. These nodes execute on-device Digital Signal Processing (DSP) using the STA/LTA (Short Term Average / Long Term Average) algorithm[cite: 1].
- **Core Backend & Processing:** A polyglot backend architecture utilizing FastAPI (Python) as the API gateway[cite: 1]. Validated data is asynchronously offloaded to a Redis Stream (`readings:stream`) and consumed by horizontally-scalable background workers via consumer groups[cite: 1]. The workers persist time-series data into a PostgreSQL/PostGIS database (provisioned as a TimescaleDB hypertable) and trigger area-scoped alerts via Redis Pub/Sub[cite: 1].
- **Client Presentation Layer:** A React Native (Expo) mobile application providing users with real-time seismograph telemetry and instantaneous critical event notifications delivered through WebSockets and native push notifications[cite: 1].

1.2. Data Plane and Control Plane

Following the v1.1.0 cloud migration, the architecture strictly separates the data and control pipelines:

- **Data Plane (Telemetry):** Flows exclusively through a HiveMQ Cloud Serverless broker on port 8883[cite: 1]. Communication is fully authenticated and TLS-encrypted[cite: 1]. A Python-based MQTT bridge (`mqtt_subscriber.py`) subscribes to the `quakeguard/telemetry` topic and forwards payloads to the internal FastAPI ingestion pipeline via HTTP POST[cite: 1].
- **Control Plane (Provisioning & Management):** Device onboarding, cryptographic handshakes, and REST retrieval operations are routed through an HTTPS tunnel to the FastAPI endpoints (e.g., `/devices/register`)[cite: 1]. In development the tunnel is a **Cloudflare quick tunnel** (`cloudflared tunnel --url http://localhost:8000`); production should use a real HTTPS domain. The ngrok free-tier edge is not used because its bot-protection terminates ESP-IDF (mbedTLS) TLS handshakes via JA3 fingerprinting **before** any HTTP header can be read, so IoT clients never reach the backend.

1.3. Key Design Principles

- **Zero-Trust Security:** Every telemetry payload must be cryptographically signed using an ECDSA (NIST256p) private key stored securely in the ESP32′s Non-Volatile Storage (NVS)[cite: 1]. The backend validates these signatures (SHA-256) and device timestamps to prevent spoofing and replay attacks[cite: 1].
- **Asynchronous Decoupling:** The ingestion API is strictly non-blocking. Validated payloads are immediately pushed to the Redis Stream, allowing the gateway to acknowledge the IoT device in milliseconds while background workers handle heavy database transactions and magnitude estimations[cite: 1].
- **Spatial Awareness:** Leveraging PostGIS, the system automatically assigns newly provisioned sensors to geographic zones using polygon containment[cite: 1]. A geohash-based Redis index (`zoneindex:<geohash>`) pre-computed at seed time provides a fast-path for coordinate-to-zone

resolution, with PostGIS `ST_Contains` as the authoritative fallback[cite: 1]. This enables targeted, geographically bounded alert broadcasting with per-area cooldowns[cite: 1].

2. Hardware & Edge Computing (ESP32-C3)

The edge layer of QuakeGuard is designed to operate on resource-constrained microcontrollers, specifically targeting the ESP32-C3 SuperMini platform (RISC-V architecture)[cite: 1]. The node interfaces with an ADXL345 digital accelerometer to acquire, filter, and analyze seismic data locally, transmitting only cryptographically signed anomaly reports to conserve bandwidth[cite: 1].

2.1. Hardware Configuration & Interfacing

Due to the specific physical layout of the ESP32-C3 SuperMini, the I2C bus is software-mapped to non-standard GPIO pins[cite: 1]:

- **SDA (Data):** Mapped to GPIO 7, requiring internal pull-up resistors[cite: 1].
- **SCL (Clock):** Mapped to GPIO 8, requiring internal pull-up resistors[cite: 1].
- **Power Constraints:** The ADXL345 is powered strictly via the 3.3V rail; applying 5V will result in immediate hardware damage[cite: 1].

The sensor operates at a 100Hz sampling rate (`ADXL345_DATARATE_100_HZ`) with a measurement range of $pm16$G (`ADXL345_RANGE_16_G`)[cite: 1]. To prevent I2C bus race conditions during startup, the sensor object is instantiated dynamically in memory only after the hardware bus is fully stabilized[cite: 1].

2.2. Digital Signal Processing (DSP) & Calibration Pipeline

Raw acceleration data undergoes multiple stages of local processing and calibration before being evaluated:

- **Static Offset Calibration:** Upon boot, the sensor performs a static 100-sample averaging sequence while the node is at rest. The calculated positional offsets are written directly to the ADXL345 hardware registers (`OFSX`, `OFSY`, `OFSZ`), inherently zeroing the sensor at the hardware level and removing physical mounting biases[cite: 1].
- **High-Pass Filter (HPF):** A digital filter (`HPF_ALPHA = 0.9f`) isolates the dynamic vibration data by subtracting the static DC component (Earth's gravity, approx. 9.81 $\frac{m}{s^2}$)[cite: 1].
- **Noise Gate:** Micro-vibrations below the empirical threshold of 0.04G are clamped to zero to prevent false positives generated by electrical noise[cite: 1].
- **Dropout Protection:** The firmware automatically drops frames reporting near-zero absolute acceleration (less than 2.0 $\frac{m}{s^2}$ prior to filtering), effectively mitigating corrupted readings caused by temporary I2C wire disconnects[cite: 1].

2.3. STA/LTA Seismic Detection Algorithm

QuakeGuard implements the industry-standard Short-Term Average / Long-Term Average (STA/LTA) algorithm[cite: 1]. The implementation utilizes a custom, memory-efficient `RingBuffer` template class in C++ to maintain rolling sums, allowing $O(1)$ average calculations[cite: 1].

- **Short-Term Window (STA):** 100 samples (1 second of telemetry), representing the immediate seismic transient[cite: 1].
- **Long-Term Window (LTA):** 1000 samples (10 seconds of telemetry), representing the background noise floor[cite: 1].
- **Trigger Condition:** An earthquake is registered when the STA/LTA ratio exceeds `1.8f`, provided the STA absolute value is also above the noise floor[cite: 1].

2.4. FreeRTOS Task Architecture

The firmware utilizes FreeRTOS to decouple time-critical sensor acquisition from network latency[cite: 1].

- **`sensorTask`:** Pinned to a high priority, it wakes exactly every 10ms to poll the ADXL345 and execute the DSP pipeline[cite: 1]. Upon detecting a seismic event, it pushes a `SeismicEvent` struct to the `eventQueue`[cite: 1].
- **`networkTask`:** Operates asynchronously at a lower priority[cite: 1]. It consumes the `eventQueue`, synchronizes the event timestamp via NTP (`pool.ntp.org`), signs the payload, and dispatches the MQTT message[cite: 1]. This design ensures that network blocking or Wi-Fi reconnects never halt the continuous monitoring of the accelerometer[cite: 1].
- **`gnssTask` (optional):** When the GNSS subsystem is compiled in, a dedicated task (priority 2) continuously feeds NMEA frames into the parser so the coordinate pipeline is always live[cite: 1].

2.5. Zero-Trust Serial Fallback (v1.2.2)

Since v1.2.2 the edge node tolerates complete MQTT/WiFi loss without losing cryptographically-attested telemetry[cite: 1]. The `networkTask` becomes a **first-available-path** dispatcher: `MQTT → USB CDC Serial → Retention Ring`[cite: 1].

- **Framing:** `SerialFallback.h` builds `[QG:FB]{...}` frames whose JSON is **byte-identical** to the MQTT payload (`value`, `sensor_id`, `device_timestamp`, `signature_hex`)[cite: 1]. The marker lets the host bridge filter CDC noise without parsing every line[cite: 1].
- **Host-aware routing:** `Serial.isConnected()` (HWCDC, `ARDUINO_USB_CDC_ON_BOOT=1`) distinguishes a real USB host from a power-only charger; with no host the event is retained rather than written to a dead port[cite: 1]. The pure routing function `decidePath(mqttReachable, usbHostPresent, timeValid)` is shared between the firmware and the host SIL test suite[cite: 1].
- **Retention & drain:** A bounded `RetentionRing<100>` holds events in FIFO order (oldest overwritten on overflow). When a path becomes reachable the ring is drained in order; each retained event is **re-signed** with the current wall time at drain so the backend ±300 s anti-replay window accepts the retransmission[cite: 1].
- **Offline wall clock:** NTP is opportunistic (`pool.ntp.org` via `configTime`). At the first successful sync the firmware anchors `epochAtSync + millis()`; subsequent timestamps remain valid after WiFi drops and no frame is emitted before `timeValid`[cite: 1].
- **Host bridge:** `firmware/tools/serial_bridge.py` reads `/dev/ttyACM0`, filters `[QG:FB]` lines via `parse_frame()`, validates the ingestion URL against SSRF (`_validate_api_url`), and POSTs to `/readings/` with `X-API-Key` — the same forwarding path as `mqtt_subscriber.py`[cite: 1]. A `test_serial_fallback.cpp` suite covers `buildSerialFrame`, `decidePath` and ring semantics on the host[cite: 1].

2.6. Diagnostic LED Indicators

The ESP32-C3 firmware drives two diagnostic LEDs to provide immediate visual feedback of the node's operational state without requiring a serial monitor[cite: 1]:

- **Boot Test:** Upon power-up, both the Blue and Red LEDs blink simultaneously twice to verify wiring integrity before the FreeRTOS tasks start[cite: 1].
- **Solid Blue:** The node is fully online, with both Wi-Fi connected and an active MQTT session established with the broker[cite: 1].
- **Single-Blinking Blue:** Wi-Fi is connected, but the MQTT broker is unreachable. The node is attempting opportunistic reconnection[cite: 1].

- **Double-Blinking Blue:** Wi-Fi connection is lost. The node is actively scanning and attempting to re-associate with the AP[cite: 1].
- **Solid Red (3-second pulse):** A seismic event (earthquake) was successfully detected by the STA/LTA algorithm. This overrides any blue LED state[cite: 1].
- **Serial Fallback Indication:** If a seismic event occurs while the Blue LED is blinking (no MQTT), the Red LED will still pulse. This specific combination (Blinking Blue + Solid Red pulse) visually confirms to the operator that the node has fallen back to writing the cryptographically-signed event to the USB CDC Serial port or parking it in the Retention Ring[cite: 1].

2.7. PCB Design & Physical Assembly (v2.0.0)

With the v2.0.0 milestone, the QuakeGuard hardware has matured from a breadboard prototype to a custom Printed Circuit Board (PCB), designed in KiCad[cite: 1]. The `hardware/` directory contains the complete schematic, PCB layout, and manufacturing Gerber files.

- **Layout Integration:** The PCB provides dedicated footprints for the ESP32-C3 SuperMini, the ADXL345 accelerometer (powered correctly at 3.3V), and the optional u-blox GNSS receiver[cite: 1].
- **Signal Integrity:** The I2C lines (SDA on GPIO 7, SCL on GPIO 8) include proper hardware pull-up resistors on the PCB to guarantee stable communication at 100Hz without relying on the weak internal MCU pull-ups[cite: 1].
- **External Interfacing:** A J4 header exposes the GNSS UART lines (RX GPIO 5, TX GPIO 4, PPS GPIO 2) and power rails, allowing modular connection of the GPS antenna for accurate time synchronization and epicentral triangulation[cite: 1].

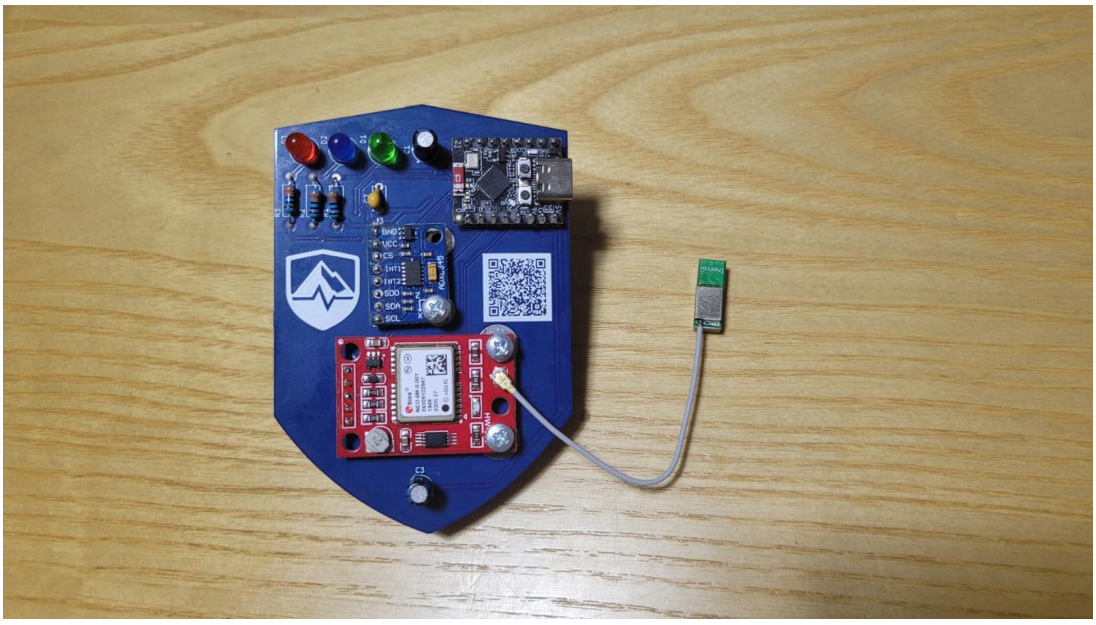

Figure 1: The QuakeGuard v2.0.0 fully assembled PCB, featuring the ESP32-C3 SuperMini, the ADXL345 accelerometer, and the u-blox GNSS module soldered into their dedicated footprints.

2.8. GNSS Subsystem & NTP Discipline (v2.0.0)

Since v1.2.1 the firmware is **GNSS-ready**: an optional module parses NMEA data from a u-blox / NEO-6M / NEO-M8N receiver over the secondary UART (RX GPIO 5, TX GPIO 4, 9600 baud on the JLCPCB — `J4-3→GPIO5`, `J4-4→GPIO4`, `J4-5→GPIO2` `PPS` for v2.0.0) at 9600 baud[cite: 1]. The module is compiled **only** when `GNSS_ENABLED=1` is present in `esp32_config.env`, so the default build stays hermetic and pulls no `TinyGPSPlus` dependency[cite: 1]. Defaults in `GnssModule.h` are `RX`

`5 / TX 4` to match the fabricated PCB; they remain overridable via `GPS_SERIAL_RX_PIN`/`TX_PIN` in `esp32_config.env` through `extra_script.py`[cite: 1].

- **Last-Known Fix Persistence:** Every reliable fix is saved into NVS (namespace `quake-gnss`, at most once per 60 seconds to limit flash wear)[cite: 1]. Provisioning therefore reports real coordinates even before the first fix after a cold boot[cite: 1].
- **Staleness Handling:** A live fix older than 10 seconds is treated as stale and the firmware falls back to the last-known value[cite: 1].
- **Provisioning Integration:** During `/devices/register`, the node reports the live GNSS fix when available, else the last-known-from-NVS fix; when neither exists it uses `GNSS_FALLBACK_LAT`/`LON` from `esp32_config.env` if defined (indoor testing), otherwise coordinates are omitted so the backend can assign the zone once placed[cite: 1].
- **NTP + PPS Discipline:** The core feature of v2.0.0. The pulse-per-second (PPS) hardware interrupt (GPIO 2) records the precise `millis()` at the start of each UTC second. The `networkTask` synchronizes via NTP and then disciplines the internal `epochAtSync` directly to the last PPS interrupt, ensuring stratum-1-like millisecond precision for all seismic telemetry[cite: 1].

2.9. Software-in-the-Loop (SIL) Validation

To ensure the theoretical STA/LTA model translates correctly to the real world, the QuakeGuard firmware edge core (`DetectionCore.h`) is tested headlessly against a Python-based Software-in-the-Loop pipeline.

- **Open Data Integration:** The pipeline dynamically queries the public INGV FDSN web service via ObsPy, fetching raw waveforms from the `IV` and `MN` open networks for specific historical baselines (e.g., L'Aquila 2009, Amatrice 2016, Emilia 2012)[cite: 1].
- **DSP Simulation:** Raw data is deconvolved to physical acceleration ($\frac{m}{s^2}$), causal bandpass-filtered (1-20 Hz), and resampled to exactly 100 Hz to simulate the ADXL345 physical output[cite: 1].
- **Algorithmic Certification:** The multidimensional calibration sweep proves the current STA/LTA threshold (`ratio=2.4`, `floor=0.02G`) yields a theoretical 80% True Positive Rate with a 0.0% False Alarm Rate against the validation dataset, demonstrating maximal rejection of distant micro-seismicity (like Salizzole at 100km) while triggering on near-fault impulses within 3 seconds of the P-wave arrival[cite: 1].

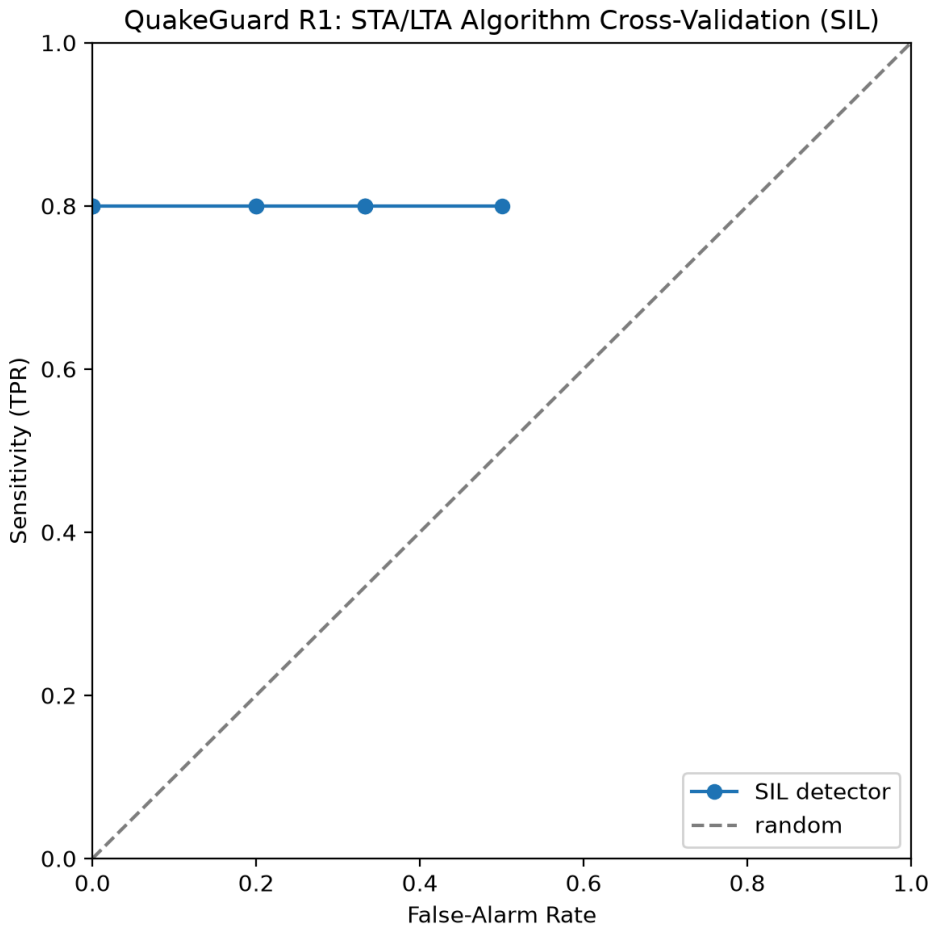

Figure 2: Receiver Operating Characteristic (ROC) curve of the edge STA/LTA algorithm. The curve reaches a hard upper bound at 0.8 (80%) True Positive Rate (Sensitivity). This limit represents a highly desirable geophysical outcome: 4 out of 5 historical events are perfectly identified without any false alarms (FAR = 0.0), while the 5th event (a weak micro-seismicity recorded at >100km distance) correctly fails to trigger the algorithm, proving the firmware's robustness against distant noise.

3. Cryptographic Security & Provisioning

QuakeGuard implements a Zero-Trust security model for its IoT edge nodes[cite: 1]. To prevent rogue devices from injecting false seismic data (spoofing) or re-transmitting old events (replay attacks), the system relies on asymmetric cryptography and strict temporal validation[cite: 1].

3.1. Cryptographic Identity (ECDSA)

Upon its first boot, the ESP32-C3 utilizes the `mbedtls` library to generate a unique Elliptic Curve Digital Signature Algorithm (ECDSA) key pair using the NIST P-256 curve (`secp256r1`)[cite: 1].

- **Private Key:** Stored securely and permanently in the device's Non-Volatile Storage (NVS)[cite: 1]. It never leaves the device and is used exclusively to sign outgoing telemetry[cite: 1].
- **Public Key:** Extracted in DER format, converted to a hexadecimal string, and acts as the unforgeable cryptographic identity of the sensor within the backend database[cite: 1].

3.2. Automated Provisioning Handshake

Before transmitting any seismic data, an unregistered sensor must complete an automated handshake with the Control Plane[cite: 1]:

1. The device sends a POST request to the `/devices/register` endpoint, providing its generated `public_key_hex`, MAC address, GPS coordinates, and a hardcoded `ENROLLMENT_TOKEN`[cite: 1].
2. The backend validates the factory enrollment token to ensure the device is authorized to join the network[cite: 1].
3. Using a geohash-based Redis fast-path index with an authoritative PostGIS fallback (`ST_Contains`), the backend spatially evaluates the provided GPS coordinates against the predefined zones and assigns the sensor to the smallest containing geographic polygon[cite: 1]. When a GNSS module is attached, the coordinates are the live fix (or the last-known fix persisted in NVS); otherwise the node reports a hardcoded placeholder until provisioned in place[cite: 1].
4. A unique `sensor_id` is returned to the device, which saves it to NVS for all future communications[cite: 1].

3.3. Payload Authentication & Integrity

When a seismic event triggers the DSP pipeline, the firmware constructs a string containing the measured magnitude and the current timestamp (format: `value:timestamp`)[cite: 1]. Under normal operation the timestamp is the NTP-synchronized wall time; when the serial fallback drains the retention ring it is the software wall clock `epochAtSync + millis()` at drain time, and the payload is **re-signed** with the current time so the backend's replay window accepts it[cite: 1]. This string is hashed via SHA-256 and signed with the device's private key[cite: 1].

The resulting JSON payload includes the data, the timestamp, and the `signature_hex` — identical on the MQTT and serial data planes (`[QG:FB]` frames carry the same JSON behind the marker)[cite: 1]. Once received by the backend, the `validate_iot_payload` dependency pipeline enforces four security gates[cite: 1]:

- **API Key Verification:** Validates the `X-API-Key` header using a constant-time comparison (`hmac.compare_digest`) to prevent timing attacks[cite: 1].
- **Sensor Status:** Confirms the sensor ID exists and is marked as `active` in the PostgreSQL database[cite: 1].
- **Anti-Replay Protection:** Compares the device's timestamp against the server's UTC time. Payloads older than a 300-second threshold are outright rejected with a `403 Forbidden` error[cite: 1].
- **Signature Verification:** The backend utilizes the Python `cryptography` library to verify the ECDSA signature against the public key associated with that sensor[cite: 1]. The implementation robustly supports both DER-encoded signatures (native to MbedTLS) and raw concatenated $r \parallel s$ signatures[cite: 1].

4. Data Plane & Message Broker (MQTT)

With the release of v1.1.0, QuakeGuard migrated its Data Plane from a local, cleartext MQTT architecture to a robust, cloud-based infrastructure[cite: 1]. This decoupling ensures that high-frequency seismic telemetry is handled by a dedicated message broker optimized for IoT, freeing the edge nodes from the overhead of HTTP round-trips[cite: 1].

4.1. HiveMQ Cloud Infrastructure

The core of the Data Plane is a HiveMQ Cloud Serverless broker[cite: 1].

- **Encrypted Transport:** All telemetry is transmitted over port 8883 using strict Transport Layer Security (TLS)[cite: 1].
- **Authentication:** Anonymous access (`allow_anonymous true`) has been deprecated[cite: 1]. The broker now requires explicit `MQTT_USERNAME` and `MQTT_PASSWORD` credentials for every connection[cite: 1].
- **Topic Topology:** All valid seismic anomalies are published to the unified `quakeguard/telemetry` topic[cite: 1].

4.2. Edge Node Implementation (Firmware)

On the ESP32-C3, the `networkTask` handles the MQTT transmission asynchronously[cite: 1].

- To support TLS on resource-constrained hardware, the firmware utilizes `WiFiClientSecure::setInsecure()` combined with the `PubSubClient` library[cite: 1].
- When a `SeismicEvent` is popped from the FreeRTOS queue, the firmware packages the JSON and fires the payload to the broker[cite: 1]. This "fire-and-forget" approach reduces transmission blocking time to milliseconds, ensuring the `sensorTask` is never starved of CPU cycles[cite: 1].

4.3. Host Serial Bridge — Second Ingestion Path (v1.2.2)

When MQTT is unreachable the host can collect `[QG:FB]` frames over USB CDC and forward them to the same ingestion pipeline. `firmware/tools/serial_bridge.py` tails `/dev/ttyACM0` (default `SERIAL_PORT`), filters lines starting with `[QG:FB]`, parses the JSON suffix and POSTs it to `/readings/` with `X-API-Key` — identical security gates as the MQTT bridge[cite: 1]. URL validation (`_validate_api_url`) restricts schemes to `http/https`, rejects embedded credentials and non-alphanumeric hostnames (SSRF guard), and `parse_frame()` returns `None` on boot-log noise so the reader never crashes on malformed lines[cite: 1]. A `dry-run` and `--stdin` mode plus the `iot-ci.yml` parser smoke test keep the bridge testable without hardware[cite: 1].

4.4. Internal MQTT Bridge Service

To securely ingest the MQTT data into the backend without exposing the FastAPI worker directly to the public internet, QuakeGuard employs a dedicated bridging microservice (`mqtt_subscriber.py`)[cite: 1].

- **Protocol Bridging:** Written in Python using the `paho.mqtt.client` (v2 API), the bridge acts as an authorized subscriber to the HiveMQ broker[cite: 1]. It initiates a secure connection using `client.tls_set(cert_reqs=ssl.CERT_REQUIRED)`[cite: 1].
- **Internal Routing:** Upon receiving a message on `quakeguard/telemetry`, the bridge wraps the payload and forwards it to the internal FastAPI ingestion endpoint (`/readings/`) via a standard HTTP POST request[cite: 1].
- **Security Injection:** The bridge injects the `X-API-Key` header into the HTTP request, acting as a trusted proxy between the external MQTT cloud and the internal Docker network[cite: 1]. If the API rejects the payload (e.g., due to an invalid cryptographic signature), the bridge logs the failure without crashing[cite: 1].

5. Backend Services & Event Processing

The QuakeGuard backend is engineered to handle massive telemetry spikes (firehosing) typical of seismic swarms[cite: 1]. To achieve this, the architecture strictly decouples data ingestion from data processing using a producer-consumer pattern mediated by Redis[cite: 1].

5.1. API Gateway & Asynchronous Ingestion

The primary entry point is a FastAPI application running behind an ASGI server (Uvicorn) within a Docker container[cite: 1].

- **Rate Limiting:** To protect against Denial of Service (DoS) and buggy edge nodes, the ingestion endpoint (`/readings/`) utilizes a sliding-window rate limiter backed by Redis sorted sets (`ZADD`, `ZREMRANGEBYSCORE`), restricting traffic to 50 requests per second per IP[cite: 1].
- **Queue Offloading:** Once a payload passes the strict cryptographic validation pipeline, the API does not block to perform database writes[cite: 1]. Instead, it immediately serializes the event and pushes it to the ingest stream via an `XADD` command, returning an HTTP `202 Accepted` to the bridge in milliseconds[cite: 1].

5.2. Redis Streams Ingestion Substrate

Since v1.2.1 the telemetry queue is a Redis **Stream** (default `readings:stream`), not a plain List. Lists cannot support consumer groups, so the previous single-worker `BRPOP` design bounded scale to one process and could lose a message on a crash mid-read[cite: 1]. Streams fix both concerns:

- **Horizontal Scale:** Any number of producers `XADD` to the stream (HTTP API, MQTT bridge, …). Any number of worker processes consume with `XREADGROUP`; Redis balances deliveries across the group, so capacity is added with `docker compose scale worker=N`[cite: 1].
- **At-Least-Once Delivery:** `XAUTOCLAIM` reclaims entries left pending by crashed consumers, so no message is lost across worker restarts[cite: 1].
- **Poisoned-Message Isolation:** Unparseable or fatally-failing messages are parked on a Dead Letter Stream (default `readings:dlq`) and acknowledged, so a poisoned heartbeat can never stall the group[cite: 1].
- **Right-Sized Backpressure:** The stream is capped with `MAXLEN ~200000` (approximate trimming); consumers read in batches of up to 64 messages with a 500ms block, and every batch is flushed in a single database transaction[cite: 1].
- **Logical Multi-Tenancy:** Every key (stream, group, DLQ, consumer prefix) is re-pointable via environment variables, so a single backend can serve multiple logical regions simply by renaming the stream[cite: 1].

5.3. Background Worker & Persistence

A decoupled Python worker (`worker.py`) runs in a continuous loop, consuming the `readings:stream` via a blocking `XREADGROUP` and reclaiming stale pending entries with `XAUTOCLAIM`[cite: 1].

- **Database Pooling:** The worker and the API share a heavily optimized SQLAlchemy connection pool targeting a PostgreSQL database[cite: 1]. The engine is configured with aggressive pooling parameters (`pool_size`, `max_overflow`) to handle high-concurrency load testing without queue exhaustion[cite: 1].
- **Batch Atomicity:** A whole stream batch is staged in-memory and persisted with a single `db.commit()`; only cooldown locks are taken per-event (atomic Redis `SET NX`). On a database error the entire batch is routed to the DLQ rather than partially applied[cite: 1].

5.4. TimescaleDB Hypertable

The `readings` table is provisioned as a TimescaleDB **hypertable** partitioned on `recorded_at`, so time-series chunks stay small and pruning stays efficient even under firehosing[cite: 1]. The provisioning module (`timescale.py`) applies the DDL best-effort and idempotently: if the connected image lacks the extension it fails closed with a warning, keeping the standard PostGIS image compatible for CI while production uses the unified TimescaleDB+PostGIS image[cite: 1]. On newer TimescaleDB releases (2.13+/3.x) the hypertable is additionally configured for columnstore access[cite: 1]. The `Reading` model uses a composite primary key `(id, recorded_at)` because TimescaleDB requires the partitioning column to be part of every unique key[cite: 1].

5.5. Geographic Zoning & Zone-Scoped Data

v1.2.1 makes PostGIS zones the source of truth for the network topology[cite: 1]:

- **Zone Management:** `GET /zones` lists the monitored polygons; `POST /zones/` creates one[cite: 1].
- **Spatial Auto-Assignment:** On provisioning, `resolve_zone` maps a device's coordinates to the smallest containing polygon. It first consults a Redis fast-path index (`zoneindex:<geohash>` → set of zone ids, precision 3), and only on a cache miss or an ambiguous multi-zone match does it run the authoritative PostGIS `ST_Contains` query ordered by polygon area. The cache is purely an optimization and can never assign the wrong zone[cite: 1].
- **Zone Detection:** `GET /zones/locate?latitude=...&longitude=...` resolves an arbitrary GPS fix into its containing monitored zone (404 when outside any polygon), powering the mobile "Detect my zone" flow[cite: 1].
- **Zone-Scoped Retrieval:** `GET /zones/{zone_id}/readings` (live per-zone telemetry for the per-zone seismograph), `DELETE /zones/{zone_id}/readings` (operational purge), and `GET /zones/{zone_id}/alerts` (confirmed alerts raised for a single area)[cite: 1].

5.6. Magnitude Estimation & Per-Area Deduplication

For every processed event, the worker performs a physical magnitude estimation based on a MyShake-style MEMS calibration approach[cite: 1]. The raw Peak Ground Acceleration (PGA) is converted using the formula[cite: 1]:

$$M_{\text{IoT}} = \log_{10}(\text{PGA}_{\text{calib}}) + b$$

Where $\text{PGA}_{\text{calib}}$ accounts for the ADXL345 scale and hardware calibration constant (`K_CALIBRATION = 1.6`), and b is an empirical offset (`B_OFFSET = 3.0`)[cite: 1]. The same normalization is shared with the mobile client so the app and the backend report identical magnitudes[cite: 1].

- **Thresholding:** If the estimated magnitude reaches or exceeds the critical threshold of 4.5, the worker triggers an `Alert` entity[cite: 1].
- **Per-Area Cooldown:** During a real earthquake, dozens of sensors in the same region will breach the threshold simultaneously. To prevent notification spam, the worker uses a Redis atomic check-and-set operation (`SET nx=True, ex=60`) keyed by the **area** — the reading's geohash region (precision 4, 39 x 19 km) when coordinates are present, else the zone — enforcing a strict 60-second cooldown per geographic area rather than globally[cite: 1]. `Reading.lat/lon` are captured at ingestion precisely to enable this fragmentation and future spatial correlation[cite: 1].
- **Outbox Pattern:** Only the first worker process that successfully acquires the Redis lock will persist the `Alert` to PostgreSQL and publish the JSON payload to the `quake_alerts` Redis Pub/Sub channel[cite: 1].

6. Mobile Client & Live Telemetry

The client-side presentation layer of QuakeGuard is a cross-platform mobile application built with React Native and the Expo framework[cite: 1]. It serves as the primary interface for users to monitor the seismic network and receive instantaneous Earthquake Early Warning (EEW) notifications[cite: 1].

6.1. Real-Time Alerting (WebSockets)

To guarantee sub-second delivery of critical alerts, the mobile app maintains a persistent, full-duplex connection to the backend via WebSockets[cite: 1].

- **Connection Management:** The `WebSocketContext` initializes a connection to the `/ws/alerts` endpoint, authenticating via a secure query parameter (`MOBILE_WS_TOKEN`)[cite: 1]. The client implements

an exponential backoff strategy for automatic reconnections up to a maximum delay of 30 seconds[cite: 1].

- **Native Haptics & Notifications:** When the WebSocket receives a payload tagged as `"CRITICAL"`, the application bypasses standard UI updates and immediately triggers native hardware APIs[cite: 1]. It executes an SOS vibration pattern (`Vibration.vibrate`) and schedules a high-priority system push notification using `expo-notifications` (`AndroidImportance.MAX`), ensuring the user is alerted even if the app is in the background[cite: 1].

6.2. Telemetry Visualization & State

The dashboard provides a live view of the network's health and recent seismic activity, relying on a robust state management architecture[cite: 1].

- **Data Fetching (TanStack Query):** Standard REST operations, such as fetching the active sensor list and the latest telemetry points (`/readings/`), are managed by React Query[cite: 1]. This provides automatic caching, background refetching (every 2 seconds for live readings), and loading state management[cite: 1].
- **Per-Zone Live Seismograph:** Since v1.2.1 the dashboard no longer mixes network-wide telemetry. A horizontal strip of zone chips (`ZoneSelector`) lets the operator pick a monitored polygon, and a `VictoryChart` (`victory-native`) renders that zone's live trace from `GET /zones/{zone_id}/readings?limit=60`[cite: 1]. The trace is anchored to the wall clock: a 60-sample / 30-second sliding window that naturally drops stale readings, uses a linear MAG scale (MIN 3.5 / MED 4.0 / ALTO 4.5) with the axis pinned left, and always renders inside the plot[cite: 1]. Raw Z-axis acceleration is converted to magnitude with the same normalization used by the backend[cite: 1].
- **Geospatial Map:** The `react-native-maps` library maps the network topology, displaying custom markers for each sensor based on their exact PostGIS coordinates and coloring them according to their active/offline status[cite: 1].

6.3. GPS Zone Detection & Alert Scoping

The Settings screen ships a "Detect my zone via GPS" action (`DETECT`)[cite: 1].

- **Flow:** `expo-location` requests foreground permission and captures a balanced-accuracy fix, then calls `GET /zones/locate?latitude=...&longitude=...` to resolve the fix into its containing monitored zone[cite: 1].
- **Outcome:** The resolved zone becomes the operator's `homeZoneId` (persisted in `usePreferencesStore`). Incoming CRITICAL alerts are gated by `ringsForZone()`: they ring, vibrate, and notify only when they match the home zone (or when no home zone is set — "ALL REGIONS")[cite: 1].
- **Error Handling:** A 404 ("coordinates outside any monitored polygon") prompts "Outside monitored area"; manual zone chips remain as a fallback[cite: 1].

6.4. Dual Theme (MIC / RESEARCH Mode)

The appearance layer ships two operator modes toggled from Settings[cite: 1]:

- **MIC MODE (dark):** `themeMode: 'dark'` — a zinc-950 tactical palette with emerald/amber/red semantic accents[cite: 1].
- **RESEARCH MODE (light):** `themeMode: 'light'` — a slate-50 "scientific paper" palette for lab analysis[cite: 1].

The active theme propagates through `useAppTheme()` to every screen, the seismograph's Victory theme, the tab bar, and the Google Map styles (Android)[cite: 1].

6.5. Resilience and Global State (Zustand)

Global application state, including alert history and user preferences, is managed using Zustand[cite: 1].

- **Alert Store:** The `useAlertStore` maintains a rolling history of the 10 most recent alerts, persisting them for the dashboard's activity log[cite: 1]. Each row can embed the matching AI Emergency Report (summary + per-item recommendations), and a dedicated `AiReportCard` renders the latest report banner inline[cite: 1].
- **Siren Audio:** On a matching CRITICAL alert the app plays a civil-defense siren (`expo-audio`, looping the bundled `alarm.wav` for a bounded 15-second window, auto-stopped and non-overlapping) alongside the native SOS `Vibration.vibrate` pattern and a MAX-priority push notification[cite: 1].
- **Offline Mode:** The `usePreferencesStore` controls a user-toggled "Offline Mode"[cite: 1]. When activated, the application cleanly and intentionally closes the active WebSocket connection and disables all React Query background polling (`enabled: !isOfflineMode`), effectively halting network traffic and conserving battery life during maintenance or low-power situations[cite: 1].

7. Deployment & Operations Guide

This section outlines the procedures for provisioning the QuakeGuard infrastructure in a local or development environment.

7.1. Prerequisites

To orchestrate the full stack, the host machine must have the following dependencies installed:

- **Backend:** Docker Engine and Docker Compose.
- **Edge (IoT):** Visual Studio Code with the PlatformIO IDE extension.
- **Mobile:** Node.js (v18+) and the Expo Go application installed on a physical smartphone.

7.2. Backend Provisioning

The backend services (PostgreSQL, TimescaleDB, Redis, FastAPI, Worker, AI-Worker, and MQTT Bridge) are fully containerized[cite: 1]. The system uses a Hybrid Network Architecture where the backend runs locally, while exposing an automated Cloudflare tunnel for external WAN traffic[cite: 1].

1. Navigate to the project root and launch the orchestrator:

```
./scripts/quakeguard_init.sh
```

This single command automates the entire infrastructure deployment:

1. Spawns `tunnel_init.sh` to negotiate a new Cloudflare HTTPS tunnel.
2. Dynamically injects the generated URL into the Mobile and Firmware `.env` configurations.
3. Opens a 3-split terminal window (using `ptyxis` or `tmux`) that simultaneously boots the Docker Compose backend (with the `--profile ai` flag for LLaMa3.2), starts the Expo bundler for the mobile client, and compiles/flashes the ESP32 firmware via PlatformIO.

7.3. Horizontal Worker Scaling

Because telemetry is consumed through Redis Streams consumer groups, the worker tier scales horizontally without reconfiguration: messages are balanced across the group automatically[cite: 1].

```
docker compose up --scale worker=N -d
```

7.4. Seismic Simulation & Stress Testing

Two companion scripts exercise the ingestion pipeline[cite: 1]:

- **`scripts/load_test.py`** — a high-concurrency End-to-End (E2E) stress test that simulates a massive seismic event through the real REST path (see the stress test section below).

- **`scripts/simulate_zone.py`** — streams synthetic per-zone readings straight through the ingestion flow, exercising the per-area cooldown fragmentation and the per-zone seismograph endpoints[cite: 1].

7.5. Edge Node & Mobile Client Orchestration

Because the orchestrator (`quakeguard_init.sh`) dynamically resolves and injects environment variables, manual configuration of the edge node and mobile client is drastically reduced.

- **Edge Node:** The ESP32 is automatically flashed via PlatformIO during the orchestrator's boot sequence (`pio run -t upload -t monitor`). The firmware strictly requires compile-time secret injection, so rebuilding ensures the node receives the latest dynamic Cloudflare provisioning URL and Fallback Coordinates (now mapped to Milan, Italy North).
- **Mobile Client:** The Expo server is launched automatically. Ensure that the smartphone running Expo Go is connected to the same Local Area Network (LAN) as the host, and scan the QR code displayed in the top-right terminal window.

7.6. Executing the Critical Stress Test

To validate the deployment, the system includes an End-to-End (E2E) stress test that simulates a massive seismic event.

From the `backend` directory, execute:

```
export API_URL="http://localhost:8000"
export NUM_SENSORS=150
export CONCURRENCY_LIMIT=50
python -m tests.stress_test
```

A successful test will conclude with the message 🏆 `SYSTEM CERTIFIED`, confirming that the rate limiter, security gates, and per-area cooldown locks are functioning nominally.

8. AI Emergency Report Service

With the release of v1.2.0, QuakeGuard introduces an on-premise AI layer that automatically generates human-readable emergency reports from confirmed seismic alerts[cite: 1]. The service is powered by a locally hosted Large Language Model (LLM) via Ollama, guaranteeing that raw telemetry — including zone coordinates and magnitude estimates — never leaves the host machine[cite: 1].

8.1. Privacy-First Local Inference

The AI pipeline is strictly self-contained and requires no external API keys or cloud LLM endpoints[cite: 1].

- **Local Ollama Service:** A dedicated `ollama` Docker container exposes the native Ollama API (`/api/generate`) on an internal Docker network[cite: 1]. On first startup the entrypoint script (`init-scripts/ollama-entrypoint.sh`) automatically pulls the configured model (`OLLAMA_MODEL`, default `llama3.2:1b`) before the service becomes healthy[cite: 1].
- **On-Premise Isolation:** Because the model runs inside the Compose network, every telemetry attribute used to compose a report stays within the host's Docker bridge network[cite: 1]. No third-party receives the data[cite: 1].
- **Model Selection:** The default 1B-parameter model is intentionally small to keep CPU and RAM footprints low while remaining fast enough to generate a report in near real-time[cite: 1]. Operators may override `OLLAMA_MODEL` (e.g., `qwen2.5:1.5b`) for higher quality output[cite: 1].

8.2. Deterministic Report Generation

Emergency messaging must never fabricate data[cite: 1]. The AI client (`ollama_client.py`) therefore constrains the model with hard guarantees[cite: 1]:

- **Sampling:** Every request is issued with `temperature: 0.0` and `top_k: 1` (greedy decoding), eliminating stochastic drift between retries[cite: 1].
- **Streaming Disabled:** Reports are generated in a single non-streaming inference pass, simplifying parsing and validation on the backend[cite: 1].
- **Strict System Prompt:** The model is instructed: *"You are an emergency response AI. Only use the provided JSON telemetry. Do not invent data."*[cite: 1]
- **Telemetry Whitelist:** The prompt is built from a fixed allowlist of fields (`zone_id`, `zone_name`, `magnitude`, `timestamp`, `sensor_count`), so the model can only reason about authenticated, persisted data[cite: 1].
- **Failure Fallback:** If the inference request fails or returns an unparseable response, the pipeline stores an explicit `"AI report unavailable."` message rather than attempting to guess[cite: 1].

8.3. Asynchronous Worker & State Machine

Report generation is fully decoupled from the alert engine via a dedicated Redis queue (`ai_report_queue`) and a separate consumer process (`ai_report_worker.py`)[cite: 1].

- **Non-Blocking Enqueue:** When the main worker (`worker.py`) persists a confirmed `Alert`, it creates an `EmergencyReport` row in `PENDING` state and pushes the alert context to `ai_report_queue` via `lpush`[cite: 1]. The alert pipeline is never blocked by LLM latency[cite: 1].
- **State Machine:** Each report transitions through a tri-state lifecycle[cite: 1]:

$$\text{PENDING} \rightarrow \text{COMPLETED}$$

$$\text{PENDING} \rightarrow \text{FAILED}$$

- **Dedicated Consumer:** The `ai-worker` container loops on a blocking `brpop`, fetches the telemetry context, invokes Ollama, and atomically flips the report state[cite: 1]. On success the worker publishes an `EMERGENCY_REPORT` payload to the `ai_reports` Redis Pub/Sub channel and commits the `COMPLETED` report with its `summary` and `recommendations`[cite: 1].
- **Dead Letter Queue:** Unrecoverable failures (missing report, persistent inference errors) push the event to `ai_report_queue_dlq` and mark the report `FAILED`[cite: 1]. The mobile app renders a "Report unavailable" badge for `FAILED` reports instead of showing partial or fabricated content[cite: 1].
- **Graceful Shutdown:** The worker traps `SIGTERM`/`SIGINT` to drain in-flight inference calls before exiting[cite: 1].

8.4. WebSocket Delivery

The mobile client receives reports through the existing real-time channel[cite: 1].

- **Channel Subscription:** The backend WebSocket broadcaster (`main.py`) subscribes to both `quake_alerts` and `ai_reports`, delivering each `EMERGENCY_REPORT` payload over the authenticated `/ws/alerts` socket[cite: 1].
- **Correlation:** The payload carries the originating `alert_id`, allowing the app to attach the report to the matching alert in its history[cite: 1].
- **REST Fallback:** A new `GET /reports/{alert_id}` endpoint exposes the persisted report for clients that reconnect after the alert (e.g., after a WebSocket drop)[cite: 1].

- **Inline Presentation:** The mobile app renders `COMPLETED` reports as a highlighted banner for the latest alert and as a dedicated card inside the alert history feed, showing the generated summary and per-item recommendations[cite: 1].

8.5. Operational Notes

- **Compose Profile:** The `ollama` and `ai-worker` services are gated behind the `ai` profile so that the default `docker compose up` remains lightweight and CI stays hermetic[cite: 1]. Enable the pipeline with `docker compose --profile ai up -d`[cite: 1].
- **Feature Flag:** The main worker only enqueues reports when `AI_REPORT_ENABLED=true` (default `false`) to avoid orphaned `PENDING` rows when the AI profile is not running[cite: 1].
- **Timeouts:** Inference requests honor `OLLAMA_TIMEOUT`; the Ollama service uses `KEEP_ALIVE` to reduce cold-start latency between alerts[cite: 1].

9. DevOps Automation & Scripting (v2.0.0)

With the release of v2.0.0, QuakeGuard introduces a "Zero-Config" orchestration pipeline designed to eliminate manual setup overhead and credential mismanagement across the distributed architecture. The `scripts/` directory provides a suite of bash tools that handle everything from dynamic cryptographic generation to multi-terminal provisioning.

9.1. The QuakeGuard Orchestrator (`quakeguard_init.sh`)

The `quakeguard_init.sh` script serves as the master entrypoint for the local development and testing environment. When executed on a fresh clone of the repository, it automates the entire lifecycle:

1. **Environment Provisioning:** It detects if the core `.env` configurations are missing and automatically invokes `generate_secrets.sh` (see below) to populate them securely.
2. **Dynamic Tunnel Negotiation:** It invokes `tunnel_init.sh` to negotiate an ephemeral Cloudflare HTTPS tunnel, instantly injecting the resulting `https://*.trycloudflare.com` URL into both the ESP32 firmware C++ build configuration and the React Native mobile client.
3. **Multi-Process Boot:** Utilizing `ptyxis` (or `tmux`), it spawns three isolated, parallel terminal sessions:
   - **Backend:** Rebuilds and launches the Docker Compose stack (including the AI-Worker profile).
   - **Mobile:** Clears the Expo cache and starts the React Native bundler.
   - **IoT Edge:** Triggers a PlatformIO `run -t upload -t monitor` command to compile the firmware with the injected dynamic URLs and flash the attached ESP32-C3 node.

9.2. Idempotent Secrets Sync (`generate_secrets.sh`)

To enforce the "Zero-Trust" architecture without manual pain, `generate_secrets.sh` generates and synchronizes cryptographic tokens (`ENROLLMENT_TOKEN`, `IOT_API_KEY`, `MOBILE_WS_TOKEN`).

- **Smart Generation:** It uses `openssl rand -hex 32` to generate secure 256-bit keys, substituting them into the respective `.env` files via regex.
- **Conflict Resolution:** If keys are modified manually and become desynchronized across the backend, mobile, or firmware, the script performs a Global Mismatch Check. It prompts the developer via an interactive `[b/m/f]` prompt to select the "Source of Truth," subsequently synchronizing the remaining systems to match the selected component.
- **Dependency Checks:** The script actively scans for missing manual configuration (such as the HiveMQ MQTT broker credentials) and forces a loud terminal warning to ensure the data plane is fully established before boot.

9.3. E2E Pipeline Stress Testing

The automation suite also includes Python-based stress testers to validate the deployment's resilience:

- **`load_test.py`:** Simulates the "Thundering Herd" effect by spawning 150 concurrent asynchronous sensors, blasting the API to validate the Redis sliding-window rate limiter and the `FastAPI` concurrency limits. A successful run finishes with a 🏆 `SYSTEM CERTIFIED` banner.
- **`simulate_zone.py`:** Streams synthetic, scaled acceleration telemetry into specific PostGIS zones to visualize live graphs on the React Native mobile app without triggering a full E2E pipeline crash.